Hands-On Generative AI

Deployment Optimization and Capstone

Day 3 - Session 4

KavinSchool

What We'll Cover

1. Deployment optimization: quantization, pruning, distillation
2. Demo: Post-Training Quantization
3. Demo: Magnitude Pruning
4. Capstone: Small Business Finance Assistant
5. Day 3 recap
6. Course recap across all three days
7. Final Q&A

Cost & Context Management

Know What You'll Spend Before You Spend It

KavinSchool

Token Counting with `tiktoken`

```python
import tiktoken

def count_tokens(text, model="gpt-4o"):
    enc = tiktoken.encoding_for_model(model)
    return len(enc.encode(text))

# Count tokens for a full conversation
def count_conversation_tokens(messages, model="gpt-4o"):
    total = 0
    for msg in messages:
        total += count_tokens(msg["content"], model)
        total += 4  # overhead per message (role, delimiters)
    total += 2  # conversation framing
    return total
```

Token Counting with `tiktoken`

Example

```
messages = [
    {"role": "system", "content": "You are a Python expert."},
    {"role": "user", "content": "Explain decorators in detail."},
]
print(f"Input tokens: {count_conversation_tokens(messages)}")
# Input tokens: ~18
```

Cost Estimation

```
# Pricing per 1M tokens (as of 2025)
PRICING = {
    "gpt-4o":       {"input": 2.50, "output": 10.00},
    "gpt-4o-mini":  {"input": 0.15, "output": 0.60},
    "gemini-flash":  {"input": 0.10, "output": 0.40},
    "claude-sonnet": {"input": 3.00, "output": 15.00},
}

def estimate_cost(input_tokens, output_tokens, model="gpt-4o"):
    p = PRICING[model]
    cost = (input_tokens * p["input"] + output_tokens * p["output"]) / 1_000_000
    return f"${cost:.4f}"

# Compare: same prompt, different models
print(estimate_cost(500, 1000, "gpt-4o"))       # $0.0113
print(estimate_cost(500, 1000, "gpt-4o-mini"))   # $0.0007
print(estimate_cost(500, 1000, "gemini-flash"))   # $0.0005
```

> **16x cost difference** between GPT-4o and GPT-4o-mini for the same prompt!

Model Selection Decision Tree

```
Is the task simple (classification, extraction, formatting)?
├── YES → Use mini/flash model (cheap, fast)
│         GPT-4o-mini, Gemini Flash, Claude Haiku
│
└── NO → Does it require deep reasoning?
    ├── YES → Use capable model
    │         GPT-4o, Claude Sonnet, Gemini Pro
    │
    └── NO → Is data privacy critical?
        ├── YES → Use open-source, self-hosted
        │         Llama 3.1, Mistral
        │
        └── NO → Start with mini, upgrade if quality is too low
```

Deployment Optimization

Smaller and faster for production

KavinSchool

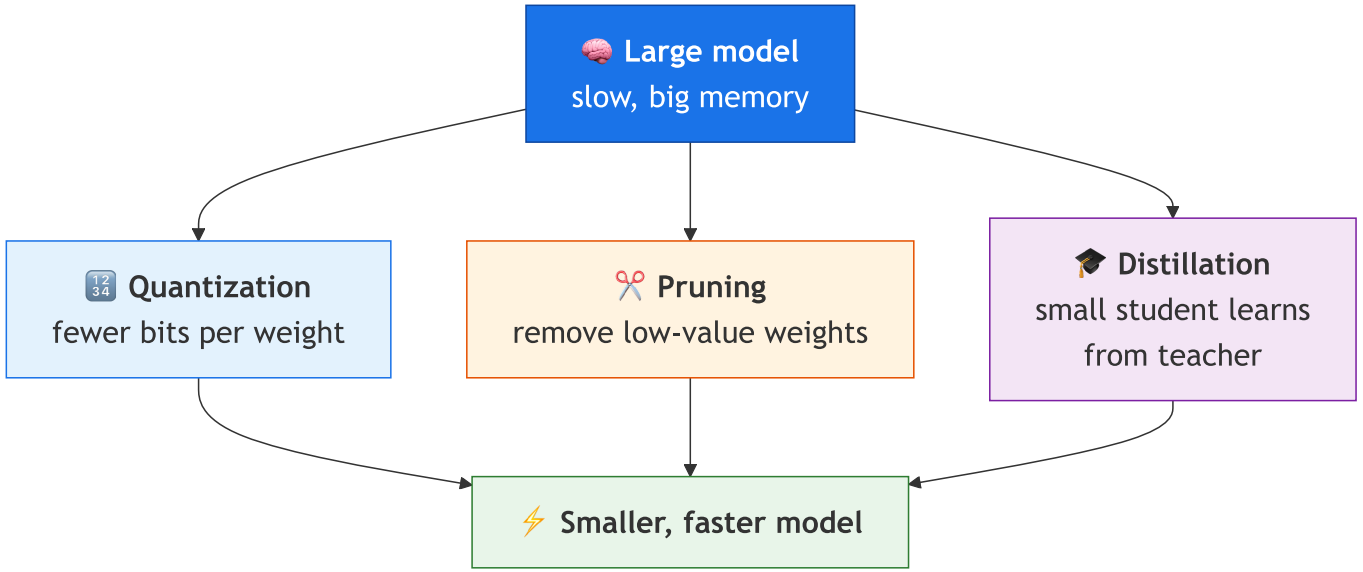
Why Optimize

A trained model is often larger and slower than a deployment needs. Optimization cuts size, memory, and latency for a small accuracy cost.

- Fit the model on the target hardware
- Lower serving cost and latency
- The techniques can be combined

> **IMPORTANT**
>
> Always evaluate your model after applying quantization or pruning. While the file size shrinks, you must ensure the accuracy remains acceptable for your specific use case.

Three Techniques



Comparing the Techniques

Technique	How	Retrain
Quantization	Fewer bits per weight	No (post-training)
Pruning	Remove low-value weights	Often a short fine-tune
Distillation	Small student learns from teacher	Yes, train the student

Reach for quantization first, then structured pruning, then distillation.

IN PRACTICE

OpenAI uses distillation to create models like GPT-4o-mini. They use the outputs and logic of the large GPT-4o model to train the much smaller, faster mini variant.

A Note on 4-bit

Dynamic int8 quantization runs on CPU and is easy to demonstrate. The 4-bit path used by QLoRA needs a CUDA GPU, so it stays concept only in this course.

- Quantize first: cheapest to apply, often enough
- Distill when you can train a permanently smaller model
- Techniques stack: distill, then quantize the student

Demo: Post-Training Quantization

Run `day3/demos/demo_quantization.py`.

- Build a small model and inspect its file size
- Run dynamic quantization to convert weights from float32 to int8
- Note the large reduction in size with minimal output drift

> Smaller model, close outputs: the quantization tradeoff.

Demo: Magnitude Pruning

Run `day3/demos/demo-pruning.py`.

- Build a small model and measure its weight sparsity
- Prune the 50% smallest-magnitude weights in each Linear layer
- Compare outputs before and after on one input

> Half the weights are zero, outputs stay close: the pruning tradeoff.

Intuit Scenario: Credit Karma Insights

Serving millions of real-time insights requires models that fit on fast hardware.

- **Base model:** Very large foundation model
- **Task:** Provide users with real-time financial health summaries
- **Challenge:** Full models take too long to run and cost too much to host
- **Outcome:** The team quantizes the model to 8-bit or distills it into a smaller version, slashing latency while maintaining accuracy

Capstone

Small Business Finance Assistant

KavinSchool

The Capstone Build

Build a RAG assistant over a mock QuickBooks and TurboTax help knowledge base with a grounded prompt.

- Chunk and embed the mock help articles
- Retrieve top-k and answer from context only
- Cite sources and refuse when context is missing
- Reuse the patterns from Lab 3.4

Capstone Stretch Goals

Pick the tiers that fit your time.

1. Add a smolagents tool for a computation (sales tax, invoice total) to make it agentic RAG
2. Run LLM-as-a-judge over a few golden Q&A pairs
3. Swap the embedding model or chunking and measure the retrieval change
4. LoRA-train a small model to route mock support tickets, reusing Lab 3.2

Capstone Tips

Keep the scope small and grounded so you finish something that works.

- Start with a handful of help articles
- Get plain RAG answering before adding an agent
- Measure one change at a time
- Log retrieved chunks so you can debug bad answers

> **TIP**
>
> If your RAG system gives wrong answers, always check the retrieved chunks first. If the right information isn't in the context window, the LLM cannot give the right answer.

Day 3 Recap

Fine-tuning, RAG, and deployment

KavinSchool

Day 3 Recap

Today you adapted models and grounded them in data.

- [x] Fine-tuned DistilBERT with transfer learning
- [x] Trained a LoRA adapter and compared sampling knobs
- [x] Built an end-to-end RAG system with evaluation
- [x] Shrank a model with quantization and planned deployment

Course Recap

Three days of generative AI

KavinSchool

Course Recap

From foundations to a working assistant.

- [x] **Day 1:** ML foundations, deep learning, generative models, diffusion
- [x] **Day 2:** NLP and embeddings, Transformers, prompting, agents
- [x] **Day 3:** fine-tuning, PEFT and LoRA, RAG, deployment

You now have the pieces to build, adapt, and ground a model.

Where to Go Next

Keep the momentum with small projects on your own data.

- Prompt first, add RAG for private facts, fine-tune last
- Measure before and after every change
- Start small, log everything, iterate

Questions?

Final Kahoot

Thanks for three days of building

KavinSchool